SCIENCE AND DEVELOPMENT JOURNAL (SDJ) — EDITORIAL PORTAL · SUBMISSION DOCUMENTATION

04 · Technical Debt Register

SDJ Editorial Portal — Technical Debt Register

Project	SDJ Editorial Portal — an editorial portal for the Science and Development Journal (SDJ), published by the College of Basic and Applied Sciences, University of Ghana
Author	Roger Koranteng Obeng (22424140)
Opened	2026-08-12 · last revised 2026-08-14
Status	Live. Entries are added at the moment debt is incurred, not reconstructed afterwards.

How this register is kept

Every entry records **Debt** → **Cause** → **Impact** → **Priority** → **Proposed resolution**, and is classified by Fowler's distinction between debt taken *deliberately* to meet a constraint and debt arising from *inadvertence*. Priority uses three levels:

- **Critical** — must be resolved before real users are admitted to the system.
- **Scheduled** — accepted now, with a named release for repayment.
- **Acceptable** — a conscious trade-off that may remain indefinitely. Each states the condition that would change that judgement.

An entry is only recorded here once the debt actually exists in the codebase. Consequences of work not yet begun are forecasts, not debt, and are kept in the relevant implementation plan instead.

Critical

TD-01 — AWS access uses root account credentials

Debt	The system is provisioned and deployed using the AWS account root user rather than a least-privilege IAM principal.
Cause	Deliberate, and decided explicitly by the owner under deadline pressure. Creating a scoped IAM deploy user was planned as the first task of deployment; it was dropped to protect the time needed to deliver the API, the frontend and the submission documents. Enabling MFA on root was separately considered and declined.
Impact	Root credentials cannot be scoped to a service, cannot be rotated per-service, and cannot be revoked without disrupting every other use of the account. Their compromise is unrecoverable within the account.
Priority	**Critical.** This is the highest-severity item in this register and the first that must be closed before the system carries real submissions.
Mitigation actually applied	The root access keys are used only from the developer's local machine and are deliberately **not** stored as GitHub Actions secrets. Deployment is therefore run locally rather than from CI. This is a meaningful reduction in exposure — CI secret stores are a far broader attack surface than a single workstation — but it is a mitigation, not a fix.
Resolution	Create an IAM deploy user restricted to App Runner, ECR, RDS, S3, Secrets Manager and CloudWatch on this project's resources; move the deployment pipeline to GitHub Actions using that user's credentials; enable MFA on root; and stop using root entirely. Estimated at under an hour.

TD-02 — A reviewer may be authorised to review their own manuscript

Debt	`Action.REVIEW` is granted on the `REVIEWER` role alone, with no per-manuscript predicate. An actor holding both `AUTHOR` and `REVIEWER` is not prevented by the policy layer from reviewing work they wrote.
Cause	Inadvertent, found by the final whole-branch review. `_OWNERSHIP_ACTIONS` covers `RESUBMIT` but the same reasoning was not applied to `REVIEW`.
Impact	The central conflict-of-interest failure for a double-blind journal — and a live gap, not a theoretical one: `POST /editorial/{code}/reviewers` creates an assignment without checking the reviewer against `author_ids`, and `POST /reviews/{code}/submit` accepts from any assigned reviewer. One partial mitigation exists: the reviewer-candidate list marks a conflicted reviewer as excluded, with the reason shown to the editor — but that is advisory at the list, not enforced at the assignment.
Priority	**Critical before real users.**
Resolution	Introduce reviewer assignment as a first-class entity; make `REVIEW` an ownership-style action predicated on an accepted assignment, and exclude actors appearing in `author_ids`. Documented in `policies.py`'s module docstring so it cannot be forgotten.

TD-15 — `GET /api/v1/people/lookup` has no rate limiting

Debt	The exact-email person lookup (`ugjcs.api.routers.people.lookup`) requires only an authenticated caller — any role — and no request throttling sits in front of it.
Cause	Deliberate scope limitation. The endpoint exists to let an author find a co-author's account id and an editor find a reviewer's, both of which need only an exact-match lookup; building it was in scope, building a rate limiter was not.
Impact	An authenticated caller can distinguish "an account exists for this address" from "no account exists" one exact guess at a time, and nothing currently bounds how many guesses per minute. Over a large-enough guessed address list this becomes account enumeration by any registered user, not merely by an anonymous attacker.
Priority	**Critical before real users.** Exact-match-only (no fuzzy/partial search) already narrows the exposure to confirming addresses the caller already suspects, but that is a mitigation, not a fix.
Resolution	Put this endpoint behind a per-caller rate limiter (e.g. a fixed-window or token-bucket check keyed on the authenticated user id, enforced at the ASGI/middleware layer or an API gateway in front of App Runner) before onboarding real users. Noted in the endpoint's own docstring so it cannot be forgotten.

TD-03 — Submitted reviews are counted, not identified

Debt	`Manuscript.record_review` increments an integer counter and never checks the supplied `reviewer_id` against an assignment, against `author_ids`, or against reviewers who have already submitted.
Cause	Deliberate scope limitation — identity-checked submission belongs with reviewer assignment as a first-class entity, which does not yet exist (the delivered candidate list ranks and excludes, but the assignment itself is a bare record).
Impact	One reviewer calling the method twice reaches the two-review quorum alone and closes the review round, so an editorial decision could rest on a single opinion.
Priority	**Critical before real users.**
Resolution	Replace `submitted_reviews: int` with `frozenset[UserId]` of reviewers who have submitted, and reject a submission from a reviewer without an accepted assignment. Documented in the method's docstring.

Scheduled

TD-04 — The audit chain has no external anchor

Debt	Hash chaining detects alteration, reordering and removal *within* the chain, but cannot detect truncation of the tail, a forged event appended through the legitimate API, or a wholly fabricated history rebuilt from the genesis hash.

Cause	Deliberate. An anchor is an operational concern outside the domain layer, and building one was not affordable in the available time.
Impact	An attacker with write access to the event store could truncate recent history, or replace it entirely, without the application detecting it. Partially mitigated: a PostgreSQL trigger rejects `UPDATE` and `DELETE` on the event table, and the foreign key refuses to delete a manuscript that has audit events.
Priority	**Scheduled** — next release after deployment.
Resolution	Publish a periodically signed checkpoint of the latest `event_hash` to storage the application cannot rewrite, and assert an expected event count at the persistence boundary. Stated explicitly in `hashchain.py`'s module docstring so no caller assumes more than the code provides.

TD-05 — Blinding does not scrub the manuscript body

Debt	`BlindedManuscript` carries `title`, `abstract` and `keywords` verbatim. Self-identifying text — an author's name in the title, or an abstract reading "extending our earlier work in [Obeng 2025]" — reaches the reviewer unchanged.
Cause	Deliberate. Reliable text redaction is a research problem; a bad redaction that leaks one name would undermine the guarantee more thoroughly than not attempting it.
Impact	Double-blind integrity depends partly on author compliance rather than entirely on the system.
Priority	**Scheduled.**
Resolution	Screening surfaces detected author-name matches to the editor before reviewers are assigned; submission guidance requires authors to anonymise their own manuscript. Automated body-text redaction is recorded as future evolution, not as a near-term fix.

TD-06 — The editorial event log has no blinded projection

Debt	Events carry `actor_id`, and decision events carry an editor's free-text rationale. There is no `BlindedEvent` type; the log is protected by authorisation policy alone (`VIEW_AUDIT` is editor-only).
Cause	Inadvertent — blinding was designed around the aggregate, and the event log was not considered as a second surface a reviewer might read.
Impact	None today, because no reviewer-facing path reads the log. The risk is that a future endpoint exposes editorial history without a projection to make the omission structural.
Priority	**Scheduled**, before any reviewer-facing audit view is built.
Resolution	Add a `BlindedEvent` projection following the same "omit the field from the type" principle as `BlindedManuscript`. Noted in `blinding.py`'s docstring.

TD-07 — No authorisation action maps to the blinded projection

Debt	`blind()` exists and `policies.can()` exists, but no `Action` connects them. An adapter serving a reviewer must remember to call `blind()`; nothing enforces it.
Cause	Inadvertent. The two modules were specified independently and the seam between them was never designed.
Impact	The double-blind guarantee rests on adapter discipline rather than on a check the system can enforce — the exact weakness the structural approach was chosen to avoid.
Priority	**Scheduled**, alongside TD-02.
Resolution	Introduce an action representing "read as an assigned reviewer" whose only successful return path is the blinded projection.

TD-08 — The tail-append capability is unverified

Debt	`hashchain.append` derives the expected sequence from the last link rather than the chain length, so it can append onto a chain tail. No test pins that capability; reverting to the older length-based form still passes the whole suite.
Cause	Inadvertent. The change was made to remove an O(n)-per-write coupling, and the test that would protect it was not added with it.
Impact	A future refactor could silently reintroduce whole-history loads on every write.
Priority	**Scheduled** — with the persistence work that first exploits tail-appending.
Resolution	A test appending onto a sliced tail whose last sequence exceeds its length, plus a repository that loads only the last link rather than the full chain.

TD-14 — Deployed infrastructure is App Runner, not the specified ECS/ALB/CloudFront

Debt	The live deployment runs the FastAPI backend on AWS App Runner behind its own `*.awsapprunner.com` TLS endpoint, rather than the ECS Fargate / Application Load Balancer / CloudFront topology designed in the design specification section 7.3.
Cause	Deliberate. Provisioning the ECS/ALB/CloudFront stack — a VPC, a target group and listener rules, a CloudFront distribution, task definitions and a service, and the IAM wiring between all of it — measured at 4–6 hours against a 48-hour budget that, at the point this trade-off was made, still owed a working API, a working frontend, and the accompanying documents.
Impact	The examiner reaches a functioning, HTTPS-verified backend, and the ECS/ALB/CloudFront design is assessed on its own architectural merits wherever the spec is read — but the two do not match. No functional capability is lost: App Runner supplies the identical trusted-TLS-without-a-registered-domain guarantee CloudFront was chosen for, over the same container image, with less to operate and less to tear down.
Priority	**Scheduled** — repayable once the document and feature backlog this budget was protecting is clear, i.e. after submission, not before.
Resolution	Provision `infra/` from the ECS/ALB/CloudFront design already specified in section 7.3, migrate the container from an App Runner source to an ECS Fargate service behind a new target group, and decommission the App Runner service once the ALB responds on the same health check. Recorded in the deployment plan's "Why this is smaller than the specification" section, so the gap is a documented, bounded decision rather than unnoticed drift.

Acceptable

TD-09 — A hybrid event log rather than full event sourcing

Debt	Current state is materialised on the manuscript row alongside the append-only event log, rather than being derived by replaying events.
Cause	Deliberate architectural trade-off, decided at design time and recorded in the specification.
Impact	Two representations of the same truth could in principle diverge. Mitigated by routing every state change through a single `_transition` method that writes both in one transaction; a grep confirms exactly one assignment to `status` in the entire aggregate.
Priority	**Acceptable.**
Condition that would change this	If projections multiply beyond the current single materialised view, or if replaying history to a past state becomes a requirement, full event sourcing with rebuildable projections becomes worth its cost.

TD-10 — The coverage gate sits at 85%, well below delivered coverage

Debt	The CI gate fails below 85% line and branch coverage, well under what the code actually achieves (90.03% across domain and application on the current commit; 100% in the domain layer).
Cause	Deliberate. The gate is a floor that prevents regression, not a target.

Impact	Coverage could fall by five points without CI objecting.
Priority	**Acceptable.**
Condition that would change this	Raising the floor is cheap and would be worthwhile if coverage ever drifts down. It is deliberately not set to 100%, because a gate at exactly the current figure makes any legitimate refactor fail the build for reasons unrelated to quality.

TD-12 — A timestamp representation reached the audit hash

Debt	Resolved, recorded because the class of defect matters. `canonical_bytes()` hashed `occurred_at.isoformat()`, which includes the UTC offset, while PostgreSQL `timestamptz` normalises any offset to UTC on storage. An event recorded at a non-UTC offset would hash one way before persistence and differently after.
Cause	Inadvertent. Every test used a UTC datetime, so the suite was structurally blind to it. Four independent reviews of the in-memory code did not find it; it was found by reasoning about the database boundary.
Impact	`verify()` would have reported tampering on an event nobody touched. A false positive is worse than no check, because it destroys confidence in every genuine detection.
Priority	**Resolved** — `canonical_bytes()` now normalises to UTC, so the offset cannot reach the digest by construction.
Resolution	Done. The lesson stands: a value that crosses a storage boundary may return in a different representation than it went in, and only an integration test or explicit canonicalisation catches it.

TD-13 — TRUNCATE bypassed the append-only guarantee

Debt	Resolved, recorded because of how it was found. A `BEFORE UPDATE OR DELETE ... FOR EACH ROW` trigger protected the editorial event log, and it was verified firing against a live database. But PostgreSQL never fires row-level triggers on `TRUNCATE`, so a single `TRUNCATE TABLE editorial_events` erased the entire audit log with no error.
Cause	Inadvertent. The trigger was written, reviewed, and empirically confirmed rejecting `UPDATE` and `DELETE`. Nobody asked whether the guarantee held for *every* class of statement that removes rows.
Impact	The system claimed a tamper-evident audit trail while leaving the single most obvious way to destroy it completely unguarded.
Priority	**Resolved** — a statement-level `BEFORE TRUNCATE ... FOR EACH STATEMENT` trigger now closes it, in both the migration and the test fixtures, with a test that fails when the trigger is removed.
Resolution	Done. The lesson: a control verified against the cases you thought of is not a control verified against the cases that matter. The question that found this was "is this claim true for every statement class?", not "does the trigger work?"

TD-11 — Coverage is a weak signal, and this project has the evidence

Debt	Not debt in the code, but a recorded limitation of the verification strategy that shapes how the other entries should be read.
Cause	Structural property of line and branch coverage.
Impact	Mutation testing during the final review found four mutations surviving a suite at 100% coverage — including deletion of the single line that makes the hash chain a chain, which left every test passing. Coverage measured that lines executed, not that behaviour was asserted.
Priority	**Acceptable** as a known limitation, provided it is compensated.
Second illustration	Coverage reported `Branch 0` for the mappers module despite branch coverage being enabled, because coverage.py does not treat an inline ternary as a branch point. "100% coverage, zero branches missed" therefore did not prove both arms of `IssueId(row.issue_id) if ... else None` had run — and the populated arm never had.

Resolution	Four gaps were closed by targeted tests once identified. Systematic mutation testing (for example `mutmut` or `cosmic-ray`) in CI is recorded as future evolution; it is the only automated technique that would have found these without a reviewer reading for intent.

Summary

Priority	Count	Entries
Critical	4	TD-01, TD-02, TD-03, TD-15
Scheduled	6	TD-04 … TD-08, TD-14
Acceptable	3	TD-09, TD-10, TD-11
Resolved, retained as a record	2	TD-12, TD-13

Repayment sequence. TD-01 before any infrastructure is provisioned. TD-02, TD-03 and TD-07 are one piece of work — they are all consequences of reviewer assignment not existing yet — and should be repaid together in the release that introduces it. TD-04 follows deployment. TD-05, TD-06 and TD-08 are independent and may be scheduled by convenience. TD-14 is repayable only after submission; it is not on the pre-viva critical path. TD-15 should be repaid before real users are admitted, alongside TD-01.

How this register was produced. Six of these fifteen entries (TD-02, TD-06, TD-07, TD-08, TD-12, TD-13) record inadvertent debt found by independent review — or, for TD-12, by reasoning about the storage boundary — in code that had already passed every automated gate: linting, strict type checking, an architecture contract, and a full test suite at 100% coverage. TD-11 records the mutation-testing evidence for why those gates could not have found them. The remaining eight are deliberate trade-offs, priced and recorded at the moment each was taken. That split is the register's most useful lesson: automated gates establish a floor, and reading code for what it *claims* is what finds the things above it.